

Program 5: Singly Linked List: Count Nodes

Aim

To write a C program that counts the number of nodes in a singly linked list.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to NULL.

The length of a linked list is not stored anywhere by default, so it is found by traversing the list and counting the nodes.

Iterative method: initialise count = 0 and increment it at every node until NULL.

Recursive method: count(NULL) = 0 and count(node) = 1 + count(node->next).

Time complexity is O(n). The iterative method uses O(1) extra space; the recursive method uses O(n) stack space.

Algorithm

1. Create the list from user input.
2. Iterative: count = 0; for each node from head to NULL, count++.
3. Recursive: if node == NULL return 0; else return 1 + count(node->next).
4. Print both counts.

C Source Code

```
/*
 * Program : Singly Linked List - Count Nodes
 *
 * THEORY:
 * - The length of a linked list is not stored anywhere by default, so it
 *   must be found by traversing the list and counting the nodes.
 * - Iterative method : counter = 0; move a pointer from head to NULL,
 *   incrementing the counter at every node.
 * - Recursive method : count(NULL) = 0; count(node) = 1 + count(node->next).
 * - Time complexity: O(n). Iterative uses O(1) extra space, recursive O(n)
 *   (call stack).
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *insertEnd(struct Node *head, int val) {
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL) return n;
    struct Node *t = head;
    while (t->next != NULL) t = t->next;
    t->next = n;
    return head;
}

int countIterative(struct Node *head) {
    int count = 0;
```

```

        for (struct Node *t = head; t != NULL; t = t->next)
            count++;
        return count;
    }

    int countRecursive(struct Node *node) {
        if (node == NULL) return 0;
        return 1 + countRecursive(node->next);
    }

    int main() {
        struct Node *head = NULL;
        int n, val;

        printf("Enter number of nodes to create: ");
        scanf("%d", &n);
        printf("Enter %d values: ", n);
        for (int i = 0; i < n; i++) {
            scanf("%d", &val);
            head = insertEnd(head, val);
        }

        printf("List: ");
        for (struct Node *t = head; t != NULL; t = t->next) printf("%d -> ", t->data);
        printf("NULL\n");

        printf("Number of nodes (iterative) = %d\n", countIterative(head));
        printf("Number of nodes (recursive) = %d\n", countRecursive(head));

        while (head != NULL) { struct Node *t = head; head = head->next; free(t); }
        return 0;
    }

```

Sample Input

Input: 6 nodes with values 11 22 33 44 55 66.

Sample Output (program run with the input above)

```

Enter number of nodes to create: Enter 6 values: List: 11 -> 22 -> 33 -> 44 -> 55 -> 66 -> NULL
Number of nodes (iterative) = 6
Number of nodes (recursive) = 6

```